PokeClub
Comprendre le code par blocs

Guide pédagogique pour une première année

L'objectif est de comprendre ce que fait un ensemble d'instructions et pourquoi ces instructions sont réunies. Chaque fonction est présentée avec son rôle, son déroulement et le détail des appels qui comptent.

Par exemple, la méthode create() est étudiée comme une action complète. On explique ce que fournit $this->defaultValues(), ce que deviennent les champs du formulaire, qui crée l'objet et qui écrit dans la base.

Le fil conducteur du projet

Le navigateur	Demande une page avec GET ou envoie un formulaire avec POST.
index.php	Démarre les outils communs et choisit l'action demandée.
Le contrôleur	Prépare les données, lance les validations et choisit la vue à afficher.
Le modèle Pokemon	Conserve les caractéristiques en mémoire et vérifie les règles métier.
Mapper et repository	Le mapper convertit les données ; le repository exécute les requêtes SQL avec PDO.
Les vues	Produisent le HTML visible : champs, messages, navigation et cartes Pokémon.

Garde ton éditeur ouvert à côté du PDF. Les extraits portent le chemin du fichier et une plage de lignes. Les répétitions de getters, de setters ou de balises de présentation sont expliquées par familles de blocs.

Le guide couvre les 33 fichiers du projet étudiés ici : pages, classes, fonctions communes, configuration publique, CSS et fichiers encore vides. Les extraits sont tirés de la version du 2 novembre 2026. Les numéros changeront si le code évolue.

Lecture conseillée : le contrôleur d'abord si tu veux comprendre create(), puis les chapitres du modèle et de la base pour suivre ses appels.

Sommaire cliquable

1. Lire les expressions PHP du projet 3
2. Démarrer et choisir la page 5
3. PokemonController : comprendre chaque fonction 7
4. Les fonctions communes et Monolog 16
5. Pokemon : l’objet et ses règles 18
6. Les énumérations de types et de natures 23
7. Le mapper et les repositories 25
8. Les équipes et leurs membres 29
9. Les vues : transformer les données en page 32
10. Les autres modèles du projet 37
11. Composer, CSS et jeu d’essai 39
12. Suivre un parcours de bout en bout 42

Les signets du lecteur PDF donnent aussi accès aux fonctions et aux sous-parties. Les chapitres se lisent séparément selon le fichier que tu as ouvert.

1. Lire les expressions PHP du projet

Ces repères servent à comprendre les blocs suivants. Le même petit nombre de mécanismes revient dans tous les fichiers.

Une classe, un objet et une méthode

Une classe décrit les données et les actions possibles. Un objet est un exemplaire construit à partir de cette classe. Une méthode est une fonction appartenant à la classe.

Exemple tiré de l'enchaînement utilisé dans index.php

```
$controller = new PokemonController($pokemonRepository, $logger);
$controller->create();
```

`new PokemonController(...)` crée un objet et exécute son constructeur. `$controller` conserve cet objet. `$controller->create()` lui demande ensuite d'exécuter son action de création.

Lire $this->defaultValues()

Expression utilisée dans PokemonController::create()

```
$values = $this->defaultValues();
```

$this	Désigne l'objet contrôleur actuellement en cours d'exécution.
->	Permet d'accéder à une propriété ou d'appeler une méthode de cet objet.
defaultValues()	Appelle une méthode du contrôleur. Les parenthèses vides indiquent qu'aucun argument n'est fourni.
Le résultat	defaultValues() retourne un tableau. Le signe = place ce tableau dans $values.

Tu peux lire l'expression ainsi : « Demande au contrôleur actuel ses valeurs initiales, puis conserve la réponse dans $values. » Les lignes de defaultValues() sont exécutées au moment de cet appel, puis le traitement reprend juste après l'appel.

Variables, tableaux et valeurs retournées

$values et $pokemon	Une variable peut contenir un tableau ou un objet. Son nom décrit ici ce qu'elle transporte.
['name' => 'Gardevoir']	Un tableau associatif relie une clé à une valeur. $values['name'] lit le nom ; $errors[] ajoute un message à la fin.
return et : void	return termine une fonction et peut fournir son résultat. : void indique que la méthode n'a pas de valeur de retour.
?Pokemon et null	?Pokemon autorise un objet Pokemon ou null. null représente ici l'absence de résultat.
=== et !==	Ces comparaisons vérifient la valeur et le type. L'entier 0 est ainsi distingué de false.
?? et ? :	a ?? b utilise b si a est absent ou null. condition ? a : b choisit entre deux valeurs.
::	S'utilise notamment pour une méthode statique ou un cas d'énumération, comme PokemonType::Psy.
public et private	Une méthode publique peut être appelée depuis index.php. Une méthode privée sert à l'intérieur de sa classe.
namespace et use	namespace donne le nom complet de la classe. use permet d'écrire son nom court dans le fichier.

Les erreurs : try, throw et catch

Le bloc `try` contient les opérations à tenter. Une instruction `throw` lance une exception et interrompt le parcours normal. Un `catch` du type attendu récupère cette exception et décide de la réponse : message du formulaire ou journalisation.

Dans ce projet, InvalidArgumentException représente notamment une saisie refusée par les règles. PDOException représente une erreur d'accès à la base. ValueError peut apparaître si une énumération reçoit une valeur inconnue. Le contrôleur transforme alors ce problème en message de validation.

Les requêtes du navigateur et la session

GET / $_GET	Une ouverture de page utilise normalement GET. Les paramètres de l'URL sont lus dans $_GET : page, id, etc.
POST / $_POST	Le formulaire method="post" envoie les champs dans $_POST. Les champs ordinaires y arrivent sous forme de texte.
$_SESSION	Conserve ici un jeton de formulaire et des confirmations entre deux requêtes. session_start() doit être exécuté avant de l'utiliser.
return / exit	return quitte la fonction ou la méthode actuelle. exit termine tout le script PHP.

2. Démarrer et choisir la page

Les fichiers de cette partie préparent les objets dont le reste de l'application a besoin. Le navigateur entre dans le projet par public/index.php.

Charger les outils communs

public/index.php | src/require/bootstrap.php

Le début d'index.php récupère une connexion PDO, un logger Monolog et un repository Pokémon. Ces objets seront ensuite transmis aux classes qui les utilisent.

Bloc du projet : public/index.php | lignes 7 à 9

```
$pdo = require __DIR__ . '/../src/require/bootstrap.php';
$logger = require __DIR__ . '/../src/require/logger.php';
$pokemonRepository = new PokemonRepository($pdo);
```

Bloc du projet : src/require/bootstrap.php | lignes 3 à 8

```
session_start();

require_once __DIR__ . '/../../vendor/autoload.php';
require_once __DIR__ . '/helpers.php';

return require __DIR__ . '/db.php';
```

`require` exécute un fichier. Si ce fichier utilise return, sa valeur peut être affectée à une variable. C'est pourquoi `$pdo = require ...bootstrap.php` récupère finalement l'objet retourné par db.php.

`require_once` charge le fichier une seule fois pendant la requête. L'autoloader Composer sait retrouver les classes ; helpers.php ajoute les fonctions partagées. session_start() prépare la session avant les formulaires.

Créer la connexion PDO

src/require/config.example.php | src/require/db.php

Le fichier de configuration décrit où se trouve MySQL. db.php utilise ces informations pour créer la connexion.

Bloc du projet : src/require/db.php | lignes 1 à 9

```
<?php

$config = require __DIR__ . '/config.local.php';

return new PDO($config['dsn'], $config['username'], $config['password'], [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
]);
```

$config	Tableau contenant le DSN, le nom du compte et le mot de passe, chargé depuis config.local.php.

new PDO(...)	Construit l'objet qui dialoguera avec MySQL. Les repositories le recevront dans leur constructeur.
ERRMODE_EXCEPTION	Une erreur SQL sera signalée par une PDOException.
FETCH_ASSOC	Une ligne lue sera un tableau dont les clés sont les noms des colonnes.
EMULATE_PREPARES	La valeur false demande l'utilisation des requêtes préparées du pilote.

`config.example.php` fournit le modèle du tableau : le DSN décrit notamment le pilote mysql, l'hôte, le port, la base pokeclub et l'encodage utf8mb4. `config.local.php` utilise cette structure avec les paramètres locaux. Ses valeurs privées restent dans ce fichier local.

Le bloc de routage dans index.php

public/index.php

Le paramètre page choisit le traitement. Ce choix est effectué avant d'afficher le HTML.

Bloc du projet : public/index.php | lignes 19 à 37

```
switch ($page) {
    case 'pokemon-create':
        $controller = new PokemonController($pokemonRepository, $logger);
        $controller->create();
        exit;

    case 'pokemon-edit':
        $controller = new PokemonController($pokemonRepository, $logger);
        $controller->edit();
        exit;

    case 'home':
        break;

    default:
        http_response_code(404);
        echo 'Page introuvable.';
        exit;
}
```

Avec `/?page=pokemon-create`, le switch appelle create(). Avec `/?page=pokemon-edit&id=42`, il appelle edit(). La valeur home quitte le switch avec break puis laisse le code de l'accueil continuer.

La vérification qui précède le switch refuse un paramètre page envoyé comme tableau. La branche default renvoie 404 pour une route inconnue. Après create() et edit(), exit évite d'exécuter aussi le code de l'accueil.

Préparer les données de l’accueil

public/index.php

Bloc du projet : public/index.php | lignes 39 à 47

```
$teamRepository = new TeamRepository($pdo, $pokemonRepository);
$team = $teamRepository->findFirst();
$pokemons = $team !== null ? $team->getPokemons() : [];
$teamTypes = [];
foreach ($pokemons as $pokemonTeam) {
    foreach ($pokemonTeam->getPokemon()->getTypes() as $type) {
        $teamTypes[$type->value] = true;
    }
}
```

TeamRepository::findFirst() charge la première équipe de la base. getPokemons() fournit ses associations PokemonTeam. Si aucune équipe n’existe, la liste affichée est vide.

La double boucle récupère les types des membres. Chaque valeur devient une clé de $teamTypes ; ajouter deux fois la même clé ne crée pas deux entrées. count($teamTypes) donnera donc le nombre de types différents.

La fin d’index.php prépare les libellés de statistiques et les variables de présentation, puis inclut home.php. Ces variables sont disponibles dans le fichier inclus.

3. PokemonController : comprendre chaque fonction

Le contrôleur orchestre le formulaire. Il reçoit des outils, prépare les champs, construit un objet valide, demande sa sauvegarde et charge la vue adaptée.

Le constructeur : recevoir les outils

src/Controller/PokemonController.php

Lorsqu’index.php crée le contrôleur, il lui fournit un PokemonRepository et un Logger Monolog.

Bloc du projet : src/Controller/PokemonController.php | lignes 16 à 23

```
private PokemonRepository $repository;
private Logger $logger;

public function __construct(PokemonRepository $repository, Logger $logger)
{
    $this->repository = $repository;
    $this->logger = $logger;
}
```

Les paramètres $repository et $logger existent pendant l’appel du constructeur. Les propriétés `$this->repository` et `$this->logger` conservent ces objets pour les autres méthodes.

Ainsi, create() peut appeler `$this->repository->save(...)` sans refaire une connexion à MySQL. Fournir les outils au constructeur s’appelle l’injection de dépendances.

defaultValues() : préparer les champs

src/Controller/PokemonController.php

Cette méthode produit le tableau initial du formulaire. Elle ne reçoit aucun argument et retourne un array.

Bloc du projet : src/Controller/PokemonController.php | lignes 143 à 163

```
private function defaultValues(): array
{
    return [
        'name' => '',
        'nature' => 'assuré',
        'picture' => '',
        'type-1' => 'normal',
        'type-2' => '',
        'item' => '',
        'move-1' => '',
        'move-2' => '',
        'move-3' => '',
        'move-4' => '',
        'hp' => '0',
        'atk' => '0',
        'def' => '0',
        'spa' => '0',
        'spd' => '0',
        'spe' => '0',
    ];
}
```

Les clés correspondent aux noms des champs HTML : name, nature, type-1, hp, etc. Les textes sont vides pour les champs à saisir. La nature assuré et le type normal sont présélectionnés ; les statistiques commencent avec le texte '0'.

Quand create() fait `$values = $this->defaultValues()`, il récupère une nouvelle valeur de tableau pour cette page. La vue affichera par exemple `$values['name']` dans le champ du nom.

create() : l’action complète de création

src/Controller/PokemonController.php

La même méthode gère l’ouverture du formulaire et son envoi. Elle ne reçoit pas d’argument ; elle prépare une réponse pour le navigateur.

Bloc du projet : src/Controller/PokemonController.php | lignes 25 à 61

```
public function create(): void
{
    $values = $this->defaultValues();
    $errors = [];
    $success = $_SESSION['pokemon_created'] ?? null;
    unset($_SESSION['pokemon_created']);
    $successMessage = 'Le Pokémon a été ajouté à la base de données. Il n’est pas encore associé
    à une équipe.';

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        verifyCsrf();

        try {
            $values = $this->readForm($values);
            $pokemon = $this->createPokemon($values);
            $this->repository->save($pokemon);

            $_SESSION['pokemon_created'] = $pokemon->getName();
            header('Location: /?page=pokemon-create', true, 303);
            exit;
        } catch (InvalidArgumentException $error) {
            $errors[] = $error->getMessage();
        } catch (PDOException $error) {
            $this->logger->error('Impossible de créer le Pokémon.');
            $errors[] = 'L’enregistrement a échoué. Réessayez dans quelques instants.';
        }

        http_response_code(422);
    }

    $pageTitle = 'Créer un Pokémon — PokeClub';
    $bodyClass = 'pokemon-create-page';
    $activePage = 'create';
    $mainContentId = 'create-main';
    $formAction = '/?page=pokemon-create';
    $submitLabel = 'Créer mon Pokémon';
    require __DIR__ . '/../View/pokemon/create.php';
}
```

D’abord : préparer ce qui sera affiché

Le contrôleur appelle defaultValues() pour préparer les champs, crée un tableau $errors vide et cherche une éventuelle confirmation dans la session. Ces trois informations correspondent aux champs, aux erreurs et au message de réussite du formulaire.

$values	Les textes que la vue placera dans les champs. Au départ, ils viennent de defaultValues(). Après un envoi ordinaire, ils viennent de readForm().
$errors	La liste des messages à afficher si la validation ou la sauvegarde échoue.
$success	Le nom du Pokémon provenant de la précédente création réussie, ou null.

unset(...)	Retire la confirmation de la session après l'avoir lue. Elle sera affichée une seule fois.

Si tu ouvres simplement la page : GET

La condition sur REQUEST_METHOD est fausse. Le bloc POST est donc sauté. Le contrôleur prépare le titre, l'adresse du formulaire et le libellé du bouton, puis inclut create.php. Cette vue inclut form.php pour afficher les champs.

Si tu appuies sur le bouton : POST

1. verifyCsrf() vérifie le jeton du formulaire. La demande est arrêtée s'il est invalide.

2. $this->readForm($values) récupère les champs envoyés. Le tableau retourné remplace les valeurs initiales.

3. $this->createPokemon($values) applique les vérifications et retourne un objet Pokemon construit avec les données acceptées.

4. $this->repository->save($pokemon) transmet cet objet au repository. Il effectue l'insertion SQL et complète l'identifiant de l'objet.

5. Le nom est conservé dans la session, puis header() envoie une redirection 303. exit termine la requête.

Le rôle précis des appels

$this->defaultValues()	Appel d'une méthode du contrôleur actuel. Résultat : un tableau de valeurs de départ.
$this->readForm($values)	Appel d'une autre méthode du même contrôleur, avec le tableau courant comme argument. Résultat : un tableau de champs préparés.
$this->createPokemon($values)	Appel du contrôleur qui construit un objet après vérification. Résultat : Pokemon, ou une exception en cas de refus.
$this->repository->save($pokemon)	Le contrôleur accède à son repository, puis appelle la méthode de cet autre objet. Cette étape écrit dans la base.
$pokemon->getName()	Appel d'un getter du modèle : il fournit le nom à conserver pour la confirmation.
require ...create.php	Exécute la vue avec les variables préparées dans la méthode. Celle-ci peut ainsi afficher les champs et les messages.

Pourquoi rediriger après une réussite ?

La redirection fait revenir le navigateur sur la page avec GET. Une actualisation ordinaire de cette nouvelle page réaffiche le formulaire au lieu de renvoyer le POST réussi. La session transporte la confirmation entre le POST et ce nouveau GET.

Si une étape échoue

Une InvalidArgumentException est récupérée par le premier catch et son message est ajouté à $errors. Une PDOException est récupérée par le second catch : un message part vers Monolog et un message simple part vers le formulaire.

Dans ces cas, le contrôleur continue jusqu'à la vue avec le statut 422. Pour un envoi ordinaire refusé après readForm(), les valeurs préparées restent dans $values et peuvent être réaffichées. En cas de réussite, exit a déjà quitté la méthode.

Exemple pour comprendre

Tu saisis « Gardevoir ». readForm() fournit « Gardevoir ». Si les autres champs sont valides, createPokemon() retourne l'objet, save() l'insère, puis la page GET affiche la confirmation avec le nom Gardevoir.

readForm() : passer des champs POST à un tableau

src/Controller/PokemonController.php

Cette méthode reçoit le tableau des champs attendus. Elle utilise ses clés pour lire les mêmes champs dans $_POST et retourne les textes préparés.

Bloc du projet : src/Controller/PokemonController.php | lignes 165 à 176

```
private function readForm(array $values): array
{
    foreach ($values as $field => $value) {
        $value = $_POST[$field] ?? '';
        if (!is_string($value)) {
            throw new InvalidArgumentException('Un champ contient une valeur invalide.');
        }
        $values[$field] = trim($value);
    }

    return $values;
}
```

La boucle parcourt les clés du formulaire. À chaque tour, `$field` peut valoir name, nature, hp, etc. La valeur POST absente est remplacée par un texte vide.

is_string() refuse une donnée envoyée sous forme de tableau. trim() retire les espaces au début et à la fin. Une fois la boucle terminée, return $values fournit le tableau à create() ou edit().

Cette méthode prépare les textes. Les statistiques sont encore des chaînes : la conversion numérique arrive dans createPokemon(). Si readForm() lance une exception, l'affectation de son résultat chez l'appelant n'a pas lieu.

Exemple pour comprendre

Pour le champ name : « Gardevoir » devient « Gardevoir ». Pour hp : le texte « 32 » devient « 32 ». Le second reste du texte à cette étape.

createPokemon() : vérifier puis construire

src/Controller/PokemonController.php

Cette méthode reçoit les valeurs préparées et doit retourner un Pokemon. Elle avance par étapes et s'arrête à la première erreur rencontrée.

1. Symfony vérifie le nom et l'image

Bloc du projet : src/Controller/PokemonController.php | lignes 180 à 196

```
$validator = Validation::createValidator();
$violations = $validator->validate($values['name'], [
    new Assert\NotBlank(message: 'Le nom du Pokémon est obligatoire.'),
    new Assert\Length(max: 255, maxMessage: 'Le nom du Pokémon doit contenir au maximum 255
    caractères.'),
]);
foreach ($violations as $violation) {
    throw new InvalidArgumentException($violation->getMessage());
}

$violations = $validator->validate($values['picture'], new Assert\Url(
    protocols: ['http', 'https'],
    requireTld: false,
    message: 'L'image doit être une URL commençant par http:// ou https://.'
));
foreach ($violations as $violation) {
    throw new InvalidArgumentException($violation->getMessage());
}
```

Validation::createValidator() fournit l'objet validateur de Symfony. validate() reçoit une valeur et une ou plusieurs contraintes. NotBlank exige un nom renseigné ; Length limite le nom à 255 caractères.

Le résultat $violations est une liste de problèmes. La boucle transforme le premier problème en InvalidArgumentException. Cette exception remontera au catch de create() ou edit().

La contrainte Url accepte un champ vide ; lorsqu'il est renseigné, elle exige ici http ou https. requireTld: false permet aussi une adresse locale comme http://localhost. Les arguments nommés message et maxMessage définissent les messages français.

2. Vérifier les champs communs et l'attaque obligatoire

Bloc du projet : src/Controller/PokemonController.php | lignes 198 à 205

```
foreach ($values as $value) {
    if (mb_strlen($value, 'UTF-8') > 255) {
        throw new InvalidArgumentException('Chaque champ doit contenir au maximum 255
    caractères.');
    }
}
if ($values['move-1'] === '') {
    throw new InvalidArgumentException('La première attaque est obligatoire.');
}
```

La boucle de longueur s'applique à tous les champs. mb_strlen() compte les caractères en UTF-8. La condition suivante impose une première attaque. Dans chaque cas de refus, throw interrompt la méthode avant la création de l'objet.

3. Transformer les statistiques en entiers

Bloc du projet : src/Controller/PokemonController.php | lignes 207 à 214

```
$points = [];
foreach (['hp', 'atk', 'def', 'spa', 'spd', 'spe'] as $stat) {
    $value = $values[$stat] === '' ? '0' : $values[$stat];
    $points[$stat] = filter_var($value, FILTER_VALIDATE_INT);
    if ($points[$stat] === false) {
        throw new InvalidArgumentException('Chaque statistique doit être un entier.');
    }
}
```

Un champ vide est interprété comme zéro. filter_var() avec FILTER_VALIDATE_INT fournit un entier si le texte est acceptable, ou false sinon.

Le test utilise === false pour distinguer un échec de l'entier 0, qui est valide. Les bornes de 0 à 32 et le total de 66 seront ensuite vérifiés par le modèle Pokemon.

4. Construire les listes de types et d'attaques

Bloc du projet : src/Controller/PokemonController.php | lignes 216 à 225

```
$types = [$values['type-1']];
if ($values['type-2'] !== '') {
    $types[] = $values['type-2'];
}
$moves = [];
foreach (['move-1', 'move-2', 'move-3', 'move-4'] as $field) {
    if ($values[$field] !== '') {
        $moves[] = $values[$field];
    }
}
```

Le premier type entre toujours dans le tableau $types. Le deuxième est ajouté seulement si son champ est renseigné. Le tableau $moves reçoit uniquement les attaques non vides, dans l'ordre des quatre champs. Le modèle pourra vérifier les doublons.

5. Construire le modèle et récupérer les valeurs inconnues

Bloc du projet : src/Controller/PokemonController.php | lignes 227 à 240

```
// Le modèle vérifie les types, la nature, les attaques et les limites des statistiques.
try {
    return new Pokemon(
        $values['name'],
        $types,
        $moves,
        $values['item'],
        $points,
        $values['nature'],
        $values['picture']
    );
} catch (ValueError $error) {
    throw new InvalidArgumentException('Choisissez une nature et des types proposés dans les
    listes.');
}
```

new Pokemon(...) exécute le constructeur du modèle. Celui-ci appelle ses setters et applique les règles métier. Si tout est accepté, return fournit le Pokemon à create() ou edit(). La sauvegarde arrivera après le retour de cette méthode.

Une valeur inconnue de type ou de nature peut provoquer une ValueError lors de la conversion en énumération. Ce catch la transforme en InvalidArgumentException avec un message adapté au formulaire. Les autres InvalidArgumentException du modèle remontent directement à create() ou edit().

edit() : charger, préremplir puis modifier

src/Controller/PokemonController.php

Cette action reprend le parcours de création, avec un identifiant et un Pokémon existant. Elle doit préserver l'identité de la ligne modifiée.

1. Retrouver le Pokémon demandé

Bloc du projet : src/Controller/PokemonController.php | lignes 65 à 86

```
$id = filter_var($_GET['id'] ?? '', FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);

if ($id === false) {
    http_response_code(400);
    echo 'Identifiant invalide.';
    return;
}

try {
    $pokemon = $this->repository->findById($id);
} catch (PDOException $error) {
    $this->logger->error('Impossible de charger le Pokémon.');
    http_response_code(503);
    echo 'Le chargement a échoué. Réessayez dans quelques instants.';
    return;
}

if ($pokemon === null) {
    http_response_code(404);
    echo 'Pokémon introuvable.';
    return;
}
```

L'identifiant vient de l'URL, par exemple id=42. filter_var() vérifie qu'il s'agit d'un entier positif. Un identifiant invalide donne 400. findById() retourne ensuite un Pokemon ou null.

Le bloc try/catch traite un échec de lecture avec 503. Le test sur null traite séparément le cas d'une lecture réussie sans Pokémon correspondant, avec 404. Dans ces cas, return quitte edit().

2. Préremplir le formulaire

Bloc du projet : src/Controller/PokemonController.php | lignes 88 à 105

```
$types = $pokemon->getTypes();
$moves = $pokemon->getMoves();
$points = $pokemon->getEffortPoints();
$values = [
    'name' => $pokemon->getName(),
    'nature' => $pokemon->getNature()->value,
    'picture' => $pokemon->getPicture(),
    'type-1' => $types[0]->value,
    'type-2' => $types[1]->value ?? '',
    'item' => $pokemon->getItem(),
    'move-1' => $moves[0] ?? '',
    'move-2' => $moves[1] ?? '',
    'move-3' => $moves[2] ?? '',
    'move-4' => $moves[3] ?? '',
];
foreach (['hp', 'atk', 'def', 'spa', 'spd', 'spe'] as $stat) {
    $values[$stat] = (string) $points[$stat];
}
```

Les getters lisent le Pokémon chargé. Les clés de $values reprennent exactement les noms des champs du formulaire. Les types et la nature sont des objets d'énumération ; ->value retrouve le texte utilisé par les options HTML.

Les attaques facultatives manquantes deviennent des chaînes vides avec ?? ''. Les statistiques deviennent des textes avec (string), pour remplir les champs. La confirmation est lue en session et affichée seulement pour l'identifiant concerné.

3. Traiter l'envoi et garder le même identifiant

Bloc du projet : src/Controller/PokemonController.php | lignes 112 à 132

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verifyCsrf();

    try {
        $values = $this->readForm($values);
        $pokemon = $this->createPokemon($values);
        $pokemon->setId($id);
        $this->repository->save($pokemon);

        $_SESSION['pokemon_updated'] = $id;
        header('Location: /?page=pokemon-edit&id=' . $id, true, 303);
        exit;
    } catch (InvalidArgumentException $error) {
        $errors[] = $error->getMessage();
    } catch (PDOException $error) {
        $this->logger->error('Impossible de modifier le Pokémon.');
        $errors[] = 'L'enregistrement a échoué. Réessayez dans quelques instants.';
    }

    http_response_code(422);
}
```

readForm() et createPokemon() sont les mêmes méthodes qu'en création. Elles produisent un nouvel objet en mémoire avec la nouvelle saisie.

L'appel décisif est `$pokemon->setId($id)`. Il donne à cet objet l'identifiant de la ligne existante. Le repository verra un identifiant présent et choisira UPDATE. Après réussite, la redirection conserve cet id dans l'URL.

4. Choisir les textes et la vue

La fin d'edit() prépare le titre « Modifier un Pokémon », le bouton « Enregistrer les modifications » et l'adresse du formulaire contenant l'id. Elle inclut ensuite edit.php. Cette vue réutilise form.php avec les valeurs préparées.

Exemple pour comprendre

Pour modifier le Pokémon 42, edit() charge d'abord 42. Après l'envoi du formulaire, le nouvel objet reçoit setId(42). save() modifie donc la ligne 42 et conserve une seule fiche.

4. Les fonctions communes et Monolog

Ces petits blocs sont appelés depuis plusieurs pages. Ils préparent l'affichage HTML, le jeton des formulaires et le journal des erreurs.

e() : préparer un texte pour le HTML

src/require/helpers.php

Cette fonction reçoit du texte et retourne sa version échappée pour l'insérer dans le HTML.

Bloc du projet : src/require/helpers.php | lignes 3 à 6

```
function e(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

htmlspecialchars() remplace les caractères qui ont un sens en HTML, notamment <, >, & et les guillemets. ENT_QUOTES inclut les deux sortes de guillemets. ENT_SUBSTITUTE remplace les séquences de caractères invalides ; UTF-8 indique l'encodage.

Dans les vues, l'expression `<?= e($values['name']) ?>` affiche ainsi le nom comme du texte. Cette fonction travaille au moment de l'affichage. La validation du nom et les requêtes SQL ont leurs propres traitements.

Exemple pour comprendre

Si le nom saisi contient Gardevoir, e() permet au navigateur d'afficher ces signes comme du texte au lieu de les interpréter comme une balise de mise en gras.

csrfToken() : fournir un jeton au formulaire

src/require/helpers.php

Cette fonction retourne le jeton de la session actuelle, en le créant s'il n'existe pas encore.

Bloc du projet : src/require/helpers.php | lignes 8 à 15

```
function csrfToken(): string
{
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }

    return $_SESSION['csrf_token'];
}
```

isset() vérifie si le jeton est déjà disponible. Sinon, random_bytes(32) crée 32 octets aléatoires ; bin2hex() les représente avec 64 caractères hexadécimaux. Le texte est conservé dans $_SESSION puis retourné.

form.php place ce résultat dans un champ caché. Le navigateur l'enverra avec les champs visibles du formulaire. La session a été démarrée dans bootstrap.php.

verifyCsrf() : vérifier l'envoi

src/require/helpers.php

Ce bloc compare le jeton du formulaire au jeton attendu dans la session avant toute sauvegarde.

Bloc du projet : src/require/helpers.php | lignes 17 à 26

```
function verifyCsrf(): void
{
    $received = $_POST['csrf_token'] ?? null;
    $expected = $_SESSION['csrf_token'] ?? null;

    if (!is_string($received) || !is_string($expected) || !hash_equals($expected, $received)) {
        http_response_code(403);
        exit('Formulaire expiré ou invalide.');
    }
}
```

La fonction récupère les deux jetons, vérifie qu'ils sont bien des chaînes et utilise hash_equals() pour les comparer. Si l'un manque ou diffère, la demande reçoit 403 et exit termine le script.

Le contrôleur appelle cette fonction avant readForm(). Une vérification réussie laisse simplement le traitement continuer, sans valeur de retour puisque la fonction est déclarée void.

logger.php : construire le journal

src/require/logger.php

Ce fichier retourne l'objet Monolog utilisé par le contrôleur.

Bloc du projet : src/require/logger.php | lignes 7 à 10

```
$logger = new Logger('pokeclub');
$logger->pushHandler(new StreamHandler(__DIR__ . '/../../var/log/pokeclub.log', Level::Error));

return $logger;
```

new Logger('pokeclub') crée le journal. pushHandler() lui ajoute un gestionnaire StreamHandler, chargé d'écrire dans le fichier var/log/pokeclub.log. Level::Error fixe le seuil : les erreurs et les niveaux plus graves sont enregistrés.

Le contrôleur reçoit cet objet dans son constructeur et appelle par exemple `$this->logger->error('Impossible de créer le Pokémon.')`. Le fichier de log conserve le message même après la fin de la requête.

Exemple pour comprendre

Si PDO refuse un enregistrement, le navigateur reçoit un message de formulaire. Le journal reçoit « Impossible de créer le Pokémon. ». Ces deux messages sont produits dans le catch du contrôleur.

5. Pokemon : l'objet et ses règles

Le contrôleur traite une demande du navigateur. Le modèle Pokemon représente les caractéristiques du Pokémon et empêche la construction de certaines valeurs invalides.

Les propriétés et le constructeur

src/Model/Pokemon.php

Les propriétés conservent l'état du Pokémon. Le constructeur reçoit les caractéristiques et passe chacune par son setter.

Bloc du projet : src/Model/Pokemon.php | lignes 8 à 27

```
{
    private ?int $id = null;
    private string $name;
    private array $types;
    private array $moves;
    private string $item;
    private array $effortPoints;
    private PokemonNature $nature;
    private string $picture;

    public function __construct(string $name, array $types, array $moves, string $item, array
    $effortPoints, string $nature, string $picture)
    {
        $this->setName($name);
        $this->setTypes($types);
        $this->setMoves($moves);
        $this->setItem($item);
        $this->setEffortPoints($effortPoints);
        $this->setNature($nature);
        $this->setPicture($picture);
    }
```

L'identifiant commence à null : l'objet n'est pas encore relié à une ligne enregistrée. Les autres propriétés sont typées : texte, tableaux ou énumération PokemonNature.

Les appels `$this->setName($name)`, `$this->setTypes($types)`, etc. exécutent les méthodes de ce même Pokémon. Cela applique les règles dès new Pokemon(...). Une exception interrompt la construction si une règle échoue.

Les getters : lire une information

src/Model/Pokemon.php

Un getter retourne une propriété. Les autres classes l'utilisent pour lire le Pokémon sans accéder directement à ses propriétés privées.

Bloc du projet : src/Model/Pokemon.php | lignes 39 à 42

```
public function getName(): string
{
    return $this->name;
}
```

getId()	Retourne un entier ou null. L'identifiant permet au repository de distinguer insertion et modification.
getName(), getItem(), getPicture()	Retournent le nom, l'objet tenu et l'adresse de l'image sous forme de texte.
getTypes()	Retourne un tableau de cas PokemonType. La vue appelle label() ; le mapper utilise ->value.
getMoves(), getEffortPoints()	Retournent les attaques et le tableau associatif des six statistiques.
getNature()	Retourne un cas PokemonNature, utilisé avec ->value ou label().

Les setters simples : id, item et picture

src/Model/Pokemon.php

setId(), setItem() et setPicture() affectent directement une valeur à leur propriété. Ils ne contiennent pas de vérification supplémentaire dans cette version.

Bloc du projet : src/Model/Pokemon.php | lignes 34 à 37

```
public function setId(int $id): void
{
    $this->id = $id;
}
```

Le paramètre $id est la valeur reçue ; `$this->id` est la propriété de l'objet. setItem() et setPicture() suivent le même modèle avec une chaîne. Ils modifient l'objet en mémoire. L'écriture SQL est réalisée par save().

setName() : refuser un nom vide

src/Model/Pokemon.php

Le setter nettoie le nom et vérifie sa règle obligatoire avant de le conserver.

Bloc du projet : src/Model/Pokemon.php | lignes 43 à 51

```
public function setName(string $name): void
{
    $name = trim($name);
    if ($name === "") {
        throw new InvalidArgumentException('Le nom du Pokémon est obligatoire');
    }

    $this->name = $name;
}
```

trim() transforme un texte composé seulement d'espaces en chaîne vide. Le test refuse alors le nom avec une InvalidArgumentException. Si le test est passé, l'affectation finale conserve le nom nettoyé.

Le contrôleur utilise aussi Symfony pour vérifier le nom du formulaire, notamment sa longueur maximale. Le modèle conserve sa règle de nom obligatoire lorsqu'un Pokemon est créé ailleurs, par exemple dans le jeu d'essai ou le mapper.

setTypes() : convertir et refuser les doublons

src/Model/Pokemon.php

Ce bloc exige un ou deux types, convertit leur texte en énumérations et refuse deux fois le même type.

Bloc du projet : src/Model/Pokemon.php | lignes 57 à 75

```
public function setTypes(array $types): void
{
    $cleanTypes = [];
    $usedTypes = [];
    if (count($types) > 2 || count($types) < 1) {
        throw new InvalidArgumentException('Le pokémon doit avoir 1 ou 2 types différents');
    }
    foreach ($types as $type) {
        $pokemonType = PokemonType::fromLocalized($type);

        if (in_array($pokemonType, $usedTypes, true)) {
            throw new InvalidArgumentException('Un pokémon ne peux avoir un double type
    identitque');
        }

        $usedTypes[] = $pokemonType;
        $cleanTypes[] = $pokemonType;
    }
    $this->types = $cleanTypes;
}
```

1. count($types) vérifie le nombre de valeurs reçues : au moins une et au maximum deux.

2. PokemonType::fromLocalized($type) transforme chaque texte en un type reconnu. Une valeur inconnue peut provoquer une ValueError.

3. in_array(..., true) vérifie strictement si ce cas a déjà été rencontré dans $usedTypes.

4. Le type accepté entre dans $usedTypes pour les vérifications et dans $cleanTypes pour le résultat. L'affectation finale conserve le tableau vérifié.

Exemple pour comprendre

Avec ['psy', 'fée'], le modèle conserve deux cas distincts. Avec ['psy', 'psy'], il lance une erreur au deuxième passage de la boucle.

setMoves() : contrôler les attaques

src/Model/Pokemon.php

Ce setter limite la liste à quatre attaques et empêche les doublons de noms, même lorsque les majuscules diffèrent.

Bloc du projet : src/Model/Pokemon.php | lignes 82 à 102

```
public function setMoves(array $moves): void
{
    $cleanMoves = [];
    $usedMoves = [];
    if (count($moves) > 4) {
        throw new InvalidArgumentException('Le pokémon ne peut apprendre que 4 attaques max');
    } else {
        foreach ($moves as $move) {
            $move = trim($move);
            $lowerMove = mb_strtolower($move, 'UTF-8');

            if (in_array($lowerMove, $usedMoves, true)) {
                throw new InvalidArgumentException('Une attaque ne peut pas être présente deux
    fois.');
            }

            $usedMoves[] = $lowerMove;
            $cleanMoves[] = $move;
        }
        $this->moves = $cleanMoves;
    }
}
```

Le bloc conserve deux représentations : le nom nettoyé, qui sera affiché, et son équivalent en minuscules, utilisé pour comparer. mb_strtolower() tient compte du texte UTF-8.

Ainsi, Psyko et psyko sont reconnus comme une même attaque. Le modèle accepte une liste de zéro à quatre attaques dans cette version ; la présence de la première attaque du formulaire est imposée par createPokemon().

setEffortPoints() : vérifier les six statistiques

src/Model/Pokemon.php

Cette méthode attend six entiers, chacun de 0 à 32, avec une somme totale d'au maximum 66.

Accepter deux formes de tableau

Bloc du projet : src/Model/Pokemon.php | lignes 119 à 127

```
$keys = ['hp', 'atk', 'def', 'spa', 'spd', 'spe'];

if (count($points) !== 6) {
    throw new InvalidArgumentException('Il faut rentrer les valeurs des 6 statistiques');
}

if (array_is_list($points)) {
    $points = array_combine($keys, $points);
}
```

Le tableau $keys fixe les noms et l'ordre des statistiques. Si $points est une liste comme [2, 32, 0, 0, 0, 32], array_combine() associe ces valeurs à hp, atk, def, spa, spd et spe. Un tableau qui possède déjà ces clés peut aussi être fourni.

Valider chaque valeur puis le total

Bloc du projet : src/Model/Pokemon.php | lignes 129 à 151

```
$cleanPoints = [];
$total = 0;

foreach ($keys as $key) {
    if (!isset($points[$key])) {
        throw new InvalidArgumentException('Il manque une statistique');
    }
    if (!is_int($points[$key])) {
        throw new InvalidArgumentException('Il faut rentrer pour chaque statistique une valeur
    numérique ');
    }
    if ($points[$key] < 0 || $points[$key] > 32) {
        throw new InvalidArgumentException('La statistique doit être comprise entre 0 et 32');
    }

    $total += $points[$key];
    $cleanPoints[$key] = $points[$key];
}

if ($total > 66) {
    throw new InvalidArgumentException('Le culmul des EV ne peux dépasser 66');
}

$this->effortPoints = $cleanPoints;
```

Pour chaque clé, le modèle vérifie la présence, le type entier et les bornes. += ajoute la valeur à $total. Le tableau $cleanPoints ne reçoit que les valeurs acceptées.

Après la boucle, le total est contrôlé. La propriété de l'objet est remplacée seulement lorsque toutes ces étapes ont réussi. Une valeur de 33 est refusée ; trois statistiques à 32 chacune sont aussi refusées car leur total dépasse 66.

setNature() : obtenir une nature reconnue

src/Model/Pokemon.php

Le setter refuse d'abord un texte vide, puis demande à PokemonNature de convertir le texte reçu.

Bloc du projet : src/Model/Pokemon.php | lignes 158 à 165

```
public function setNature(string $nature): void
{
    $normalized = mb_strtolower(trim($nature));
    if ($normalized === "") {
        throw new InvalidArgumentException('Un pokémon doit avoir une nature');
    }
    $this->nature = PokemonNature::fromLocalized($nature);
}
```

La variable $normalized sert ici à tester si le texte est vide après préparation. fromLocalized() fait ensuite sa propre normalisation et sa traduction éventuelle. La propriété nature conserve finalement un cas d'énumération, pas une simple chaîne.

6. Les énumérations de types et de natures

Une énumération est une liste fermée de cas nommés. Elle permet de manipuler des valeurs reconnues et de retrouver leur texte pour le formulaire ou la base.

Les cas de PokemonType et PokemonNature

src/Model/PokemonType.php | src/Model/PokemonNature.php

Les déclarations enum ...: string associent chaque cas à une valeur texte.

Bloc du projet : src/Model/PokemonType.php | lignes 5 à 10

```
enum PokemonType: string
{
    case Normal = 'normal';
    case Eau = 'eau';
    case Feu = 'feu';
    case Plante = 'plante';
```

PokemonType contient les 18 types présents dans le fichier, jusqu'à Poison. PokemonNature contient les 25 natures, d'Assure à Timide. Les autres lignes case suivent exactement le même principe que cet extrait.

PokemonType::Psy désigne un cas. `PokemonType::Psy->value` fournit le texte psy. cases() donne le tableau de tous les cas, ce qui permet aux vues de construire les listes déroulantes automatiquement.

label() : préparer le texte affiché

src/Model/PokemonType.php | src/Model/PokemonNature.php

Bloc du projet : src/Model/PokemonType.php | lignes 26 à 29

```
public function label(): string
{
    return mb_convert_case($this->value, MB_CASE_TITLE, 'UTF-8');
}
```

Dans un cas d'énumération, `$this->value` est la chaîne associée. mb_convert_case() avec MB_CASE_TITLE prépare les initiales majuscules en UTF-8 : psy devient Psy. La méthode label() de PokemonNature suit le même modèle.

fromLocalized() : retrouver un type

src/Model/PokemonType.php

Bloc du projet : src/Model/PokemonType.php | lignes 31 à 35

```
public static function fromLocalized(string $input): self
{
    $normalized = mb_strtolower(trim($input), 'UTF-8');
    return self::from($normalized);
}
```

Cette méthode est statique : on l'appelle sur la classe avec ::, sans construire un objet au préalable. Elle retire les espaces, passe en minuscules et appelle self::from(). self désigne ici l'énumération PokemonType.

from() cherche le cas dont la valeur correspond au texte. Si aucune valeur ne correspond, PHP lance une ValueError. Le type de retour self annonce un cas de cette même énumération.

fromLocalized() : traduire puis retrouver une nature

src/Model/PokemonNature.php

Bloc du projet : src/Model/PokemonNature.php | lignes 64 à 70

```
public static function fromLocalized(string $input): self
{
    $normalized = mb_strtolower(trim($input), 'UTF-8');
    $canonical = self::LOCALIZED_MAP[$normalized] ?? $normalized;

    return self::from($canonical);
}
```

LOCALIZED_MAP est un tableau constant qui associe certains noms anglais à leurs valeurs françaises. Après la préparation du texte, l'opérateur ?? conserve le nom normalisé lorsqu'aucune traduction n'est nécessaire.

Par exemple, adamant est associé à rigide et modest à modeste. La valeur canonique obtenue est transmise à from(). Le constructeur de Pokemon utilise cette méthode pour conserver une nature reconnue.

7. Le mapper et les repositories

Ces blocs relient les objets PHP à la base de données. Le mapper fait les conversions ; les repositories utilisent PDO pour exécuter les requêtes.

PokemonMapper::fromRow() : reconstruire un objet

src/Mapper/PokemonMapper.php

La méthode reçoit une ligne SQL associative et retourne un Pokemon complet, avec son identifiant.

Bloc du projet : src/Mapper/PokemonMapper.php | lignes 12 à 31

```
$types = [$row['pokemon_type1']];
if ($row['pokemon_type2'] !== null) {
    $types[] = $row['pokemon_type2'];
}

$moves = [];
foreach (['pokemon_move1', 'pokemon_move2', 'pokemon_move3', 'pokemon_move4'] as $column) {
    if ($row[$column] !== null) {
        $moves[] = $row[$column];
    }
}

$points = [
    'hp' => (int) $row['pokemon_hp'],
    'atk' => (int) $row['pokemon_atk'],
    'def' => (int) $row['pokemon_def'],
    'spa' => (int) $row['pokemon_spa'],
    'spd' => (int) $row['pokemon_spd'],
    'spe' => (int) $row['pokemon_spe'],
];
```

La base répartit les types et les attaques dans plusieurs colonnes. Le mapper les regroupe dans les tableaux attendus par le modèle et ignore les colonnes facultatives qui valent null.

Les statistiques sont converties en entiers avec (int). Le constructeur de Pokemon attend ces entiers ; ils pourront ainsi passer les vérifications is_int() du modèle.

Bloc du projet : src/Mapper/PokemonMapper.php | lignes 33 à 44

```
$pokemon = new Pokemon(
    $row['pokemon_name'],
    $types,
    $moves,
    $row['pokemon_item'] ?? '',
    $points,
    $row['pokemon_nature'],
    $row['pokemon_picture']
);
$pokemon->setId((int) $row['id']);

return $pokemon;
```

Le new Pokemon(...) construit l'objet avec les caractéristiques préparées et déclenche les règles du modèle. setId() lui ajoute ensuite l'identifiant SQL. return fournit cet objet au repository, qui le transmettra à son appelant.

PokemonMapper::toData() : préparer l’écriture

src/Mapper/PokemonMapper.php

Cette méthode reçoit un Pokemon et retourne les valeurs nommées attendues par les requêtes INSERT ou UPDATE.

Bloc du projet : src/Mapper/PokemonMapper.php | lignes 47 à 71

```
public function toData(Pokemon $pokemon): array
{
    $types = $pokemon->getTypes();
    $moves = $pokemon->getMoves();
    $points = $pokemon->getEffortPoints();

    return [
        'name' => $pokemon->getName(),
        'type_1' => $types[0]->value,
        'type_2' => isset($types[1]) ? $types[1]->value : null,
        'item' => $pokemon->getItem(),
        'picture' => $pokemon->getPicture(),
        'move_1' => $moves[0] ?? null,
        'move_2' => $moves[1] ?? null,
        'move_3' => $moves[2] ?? null,
        'move_4' => $moves[3] ?? null,
        'hp' => $points['hp'],
        'atk' => $points['atk'],
        'def' => $points['def'],
        'spa' => $points['spa'],
        'spd' => $points['spd'],
        'spe' => $points['spe'],
        'nature' => $pokemon->getNature()->value,
    ];
}
```

Les getters récupèrent les tableaux de l’objet. Les cas de types deviennent des chaînes avec ->value. Un deuxième type ou une attaque facultative absente devient null.

La fin du tableau ajoute les six statistiques et la chaîne de la nature. Les clés name, type_1, hp, nature, etc. correspondent aux paramètres :name, :type_1, :hp et :nature utilisés dans le repository.

Le constructeur de PokemonRepository

src/Repository/PokemonRepository.php

Le repository conserve PDO et crée le mapper utilisé pour ses conversions.

Bloc du projet : src/Repository/PokemonRepository.php | lignes 14 à 18

```
public function __construct(PDO $pdo)
{
    $this->pdo = $pdo;
    $this->mapper = new PokemonMapper();
}
```

La connexion est fournie depuis index.php. Le repository peut ensuite appeler `$this->pdo->prepare(...)` pour préparer une requête et `$this->mapper->fromRow(...)` pour convertir son résultat.

findById() : chercher une fiche

src/Repository/PokemonRepository.php

La méthode reçoit un identifiant entier et retourne un Pokemon ou null.

Bloc du projet : src/Repository/PokemonRepository.php | lignes 35 à 46

```
public function findById(int $id): ?Pokemon
{
    $statement = $this->pdo->prepare('SELECT * FROM pokemon WHERE id = :id');
    $statement->execute(['id' => $id]);
    $row = $statement->fetch();

    if ($row === false) {
        return null;
    }

    return $this->mapper->fromRow($row);
}
```

prepare(...)	Prépare le texte SQL. :id est un emplacement nommé qui recevra une valeur.
execute(['id' => $id])	Exécute la requête en liant le paramètre à l'identifiant reçu.
fetch()	Récupère une ligne associative, ou false si la recherche n'a aucun résultat.
return null	Permet au contrôleur de traiter le cas « Pokémon introuvable ».
fromRow($row)	Transforme la ligne trouvée en objet Pokemon avant le retour.

findAll() et findByName() : chercher plusieurs fiches

src/Repository/PokemonRepository.php

Ces deux méthodes retournent toujours un tableau d'objets Pokemon. Il peut être vide si aucune ligne ne correspond.

Bloc du projet : src/Repository/PokemonRepository.php | lignes 48 à 60

```
public function findAll(): array
{
    $statement = $this->pdo->prepare('SELECT * FROM pokemon ORDER BY id');
    $statement->execute();
    $rows = $statement->fetchAll();

    $pokemons = [];
    foreach ($rows as $row) {
        $pokemons[] = $this->mapper->fromRow($row);
    }

    return $pokemons;
}
```

findAll() lit toutes les lignes triées par id. fetchAll() fournit une liste de lignes SQL. La boucle convertit chaque ligne avec le mapper, puis retourne la liste des objets.

findByName($name) suit le même parcours après trim($name), avec une condition WHERE pokemon_name = :name. Il recherche le nom exact et retourne une liste, car plusieurs configurations peuvent porter ce nom. La méthode findAll() est disponible pour la future page de collection.

save() : insérer ou modifier

src/Repository/PokemonRepository.php

Cette méthode reçoit un Pokemon. Elle demande d'abord au mapper le tableau $data, puis choisit son SQL selon l'identifiant.

Bloc du projet : src/Repository/PokemonRepository.php | lignes 67 à 79

```
    $sql = 'INSERT INTO pokemon (
    pokemon_name, pokemon_type1, pokemon_type2, pokemon_item, pokemon_picture,
    pokemon_move1, pokemon_move2, pokemon_move3, pokemon_move4,
    pokemon_hp, pokemon_atk, pokemon_def, pokemon_spa, pokemon_spd, pokemon_spe, pokemon_nature
) VALUES (
    :name, :type_1, :type_2, :item, :picture,
    :move_1, :move_2, :move_3, :move_4,
    :hp, :atk, :def, :spa, :spd, :spe, :nature
)';

    $statement = $this->pdo->prepare($sql);
    $statement->execute($data);
    $pokemon->setId((int) $this->pdo->lastInsertId());
```

Si getId() retourne null, la ligne n'existe pas encore pour cet objet : le repository prépare INSERT. Les listes de colonnes et de paramètres sont dans le même ordre. execute($data) fournit les valeurs à leurs paramètres.

Après l'insertion, lastInsertId() récupère l'identifiant créé par la base. Le repository l'attribue au même objet Pokemon avec setId(). Le contrôleur conserve donc un objet qui connaît maintenant son id.

Bloc du projet : src/Repository/PokemonRepository.php | lignes 81 à 103

```
$data['id'] = $pokemon->getId();

$sql = 'UPDATE pokemon SET
pokemon_name = :name,
pokemon_type1 = :type_1,
pokemon_type2 = :type_2,
pokemon_item = :item,
pokemon_picture = :picture,
pokemon_move1 = :move_1,
pokemon_move2 = :move_2,
pokemon_move3 = :move_3,
pokemon_move4 = :move_4,
pokemon_hp = :hp,
pokemon_atk = :atk,
pokemon_def = :def,
pokemon_spa = :spa,
pokemon_spd = :spd,
pokemon_spe = :spe,
pokemon_nature = :nature
WHERE id = :id';

$statement = $this->pdo->prepare($sql);
$statement->execute($data);
```

Si un identifiant est déjà présent, la branche else ajoute id à $data et prépare UPDATE. Les affectations intermédiaires mettent à jour les mêmes caractéristiques que l'insertion.

WHERE id = :id limite l'UPDATE à cette fiche. C'est pourquoi edit() attribue l'id existant avant save(). La méthode est déclarée void : son effet est l'écriture en base et, lors d'un INSERT, la mise à jour de l'identifiant de l'objet.

Exemple pour comprendre

Un objet avec id = null conduit à INSERT et reçoit ensuite son nouvel id. Un objet avec id = 42 conduit à UPDATE ... WHERE id = 42.

8. Les équipes et leurs membres

Une équipe contient des associations PokemonTeam. Chaque association relie un Pokémon, une équipe et un rôle. Le code actuel sait lire les équipes et modifier leurs objets en mémoire.

Team : constructeur et accès aux propriétés

src/Model/Team.php

Le constructeur reçoit le nom. L'identifiant commence à null et le tableau des membres commence vide.

getId()/setId() et getName()/setName() lisent ou modifient les propriétés correspondantes. Le setter de nom affecte directement le texte dans cette version. Les règles de composition se trouvent dans addPokemon() et removePokemon().

addPokemon() : ajouter un membre en mémoire

src/Model/Team.php

Bloc du projet : src/Model/Team.php | lignes 37 à 50

```
public function addPokemon(Pokemon $pokemon, string $role): void
{
    if (count($this->pokemons) >= 6) {
        throw new InvalidArgumentException('Vous ne pouvez composez une équipe que de 6 pokémon
    distinct');
    }

    foreach ($this->pokemons as $pokemonTeam) {
        if ($pokemonTeam->getPokemon() === $pokemon) {
            throw new InvalidArgumentException('Ce pokémon est déjà présent');
        }
    }

    $this->pokemons[] = new PokemonTeam($pokemon, $this, $role);
}
```

La première condition interdit un septième membre. La boucle cherche ensuite si le même objet Pokemon est déjà présent. Avec des objets, === vérifie l'identité de l'objet, et non l'égalité de leurs identifiants SQL.

L'ajout crée un PokemonTeam. Le paramètre $this désigne ici l'équipe actuelle ; le nouvel objet relie donc le Pokémon reçu à cette équipe avec le rôle fourni. [] ajoute cette association au tableau.

getPokemons() et removePokemon()

src/Model/Team.php

Bloc du projet : src/Model/Team.php | lignes 57 à 66

```
public function removePokemon(Pokemon $pokemon): void
{
    foreach ($this->pokemons as $index => $pokemonTeam) {
        if ($pokemonTeam->getPokemon() === $pokemon) {
            unset($this->pokemons[$index]);
            return;
        }
    }
    throw new InvalidArgumentException('Aucun pokémon ne correspond à votre demande');
}
```

removePokemon() parcourt les associations avec leur indice, trouve celle du même objet Pokemon et la retire avec unset(). return arrête la recherche dès que le retrait est effectué. Si rien n'a correspondu, la méthode lance une erreur.

getPokemons() utilise array_values() pour retourner les associations avec des indices consécutifs, y compris après une suppression. Ces opérations changent la composition en mémoire ; une méthode de sauvegarde d'équipe reste à écrire pour les enregistrer en base.

PokemonTeam : une association avec un rôle

src/Model/PokemonTeam.php

Cet objet conserve les deux objets reliés et le rôle propre à leur association.

Bloc du projet : src/Model/PokemonTeam.php | lignes 13 à 18

```
public function __construct(Pokemon $pokemon, Team $team, string $role)
{
    $this->pokemon = $pokemon;
    $this->team = $team;
    $this->setRole($role);
}
```

Le constructeur conserve $pokemon et $team. Il appelle setRole() pour vérifier le rôle avant de le conserver. getPokemon(), getTeam() et getRole() permettent ensuite de retrouver ces trois informations.

setRole() refuse exactement la chaîne vide et lance une InvalidArgumentException. Il ne fait pas de trim() dans cette version : la règle porte sur ce texte tel qu'il est reçu.

Exemple pour comprendre

Gardevoir peut avoir le rôle « support » dans une équipe. Cette information appartient à PokemonTeam ; elle ne modifie pas les caractéristiques de l'objet Pokemon.

TeamRepository : recevoir les outils de lecture

src/Repository/TeamRepository.php

Le constructeur reçoit PDO et un PokemonRepository. Le premier lit les tables ; le second reconstruit les Pokémon membres.

Bloc du projet : src/Repository/TeamRepository.php | lignes 19 à 28

```
public function findFirst(): ?Team
{
    $id = $this->pdo->query('SELECT id FROM team ORDER BY id LIMIT 1')->fetchColumn();

    if ($id === false) {
        return null;
    }

    return $this->findById((int) $id);
}
```

findFirst() récupère le plus petit id de la table team avec ORDER BY id LIMIT 1. fetchColumn() lit la première colonne du résultat. S'il n'y a aucune ligne, la méthode retourne null ; sinon elle réutilise findById().

findById() : reconstruire l'équipe complète

src/Repository/TeamRepository.php

Bloc du projet : src/Repository/TeamRepository.php | lignes 32 à 46

```
$statement = $this->pdo->prepare('SELECT * FROM team WHERE id = :id');
$statement->execute(['id' => $id]);
$row = $statement->fetch();

if ($row === false) {
    return null;
}

$team = new Team($row['name']);
$team->setId((int) $row['id']);

$statement = $this->pdo->prepare(
    'SELECT pokemon_id, role FROM pokemon_team WHERE team_id = :team_id ORDER BY pokemon_id'
);
$statement->execute(['team_id' => $id]);
```

La première requête lit la ligne de team. Le repository crée un Team avec son nom et lui attribue son id. La deuxième requête lit dans pokemon_team les identifiants des membres et leurs rôles pour cette équipe.

Bloc du projet : src/Repository/TeamRepository.php | lignes 48 à 58

```
foreach ($statement->fetchAll() as $pokemonRow) {
    $pokemon = $this->pokemonRepository->findById((int) $pokemonRow['pokemon_id']);

    if ($pokemon === null) {
        throw new RuntimeException("Un Pokémon de l'équipe est introuvable.");
    }

    $team->addPokemon($pokemon, $pokemonRow['role']);
}

return $team;
```

Pour chaque association, PokemonRepository::findById() fournit un objet Pokemon complet. Le repository vérifie qu'il a été retrouvé, puis addPokemon() le relie à l'équipe avec son rôle.

Le retour final est un objet Team avec sa composition reconstruite. La sauvegarde d'une composition et son formulaire d'édition ne sont pas encore présents dans les routes actuelles.

9. Les vues : transformer les données en page

Les vues utilisent les variables préparées par index.php ou le contrôleur. Leur rôle est d'afficher les données, les champs et les messages en HTML.

header.php et footer.php : encadrer la page

src/View/header.php | src/View/footer.php

Ces deux fichiers fournissent le début et la fin communs du document HTML.

Bloc du projet : src/View/header.php | lignes 5 à 15

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="PokeClub, votre créateur d'équipe Pokémon.">
    <title><?= htmlspecialchars($pageTitle ?? 'PokeClub — Créateur d'équipe Pokémon',
    ENT_QUOTES, 'UTF-8') ?></title>
    <link rel="stylesheet" href="/assets/css/tailwind.css">
</head>

<body class="<?= htmlspecialchars($bodyClass ?? 'pokemon-page', ENT_QUOTES, 'UTF-8') ?>">
    <?php if (isset($mainContentId)) { ?>
        <a class="create-skip-link" href="#<?= htmlspecialchars($mainContentId, ENT_QUOTES,
    'UTF-8') ?>">Aller au contenu</a>
    <?php } ?>
```

Le titre, la classe du body et le lien d'accès rapide viennent des variables $pageTitle, $bodyClass et $mainContentId. Les valeurs par défaut permettent au header de fonctionner même lorsqu'une de ces variables n'a pas été préparée.

Le lien stylesheet charge le CSS généré. La navigation utilise $activePage pour ajouter aria-current="page" sur l'entrée correspondante. Le logo est un dessin SVG décoratif. footer.php ferme body et html.

Pourquoi une vue voit-elle les variables du contrôleur ?

Quand le contrôleur exécute `require ...create.php`, le fichier inclus peut utiliser les variables locales déjà définies. create.php inclut à son tour form.php. C'est ainsi que ce formulaire accède à $values, $errors, $success ou $submitLabel.

create.php et edit.php : disposer les éléments

src/View/pokemon/create.php | src/View/pokemon/edit.php

Les deux pages utilisent le même formulaire. Le contrôleur choisit les valeurs, les textes et l'adresse d'envoi adaptés à son action.

Bloc du projet : src/View/pokemon/edit.php | lignes 1 à 16

```
<?php require __DIR__ . '/../header.php'; ?>

<main id="edit-main" class="create-container create-main">
    <a class="create-back" href="/">Retour à mon équipe</a>
    <div class="create-intro">
        <p class="create-eyebrow">AJUSTEZ VOTRE STRATÉGIE</p>
        <h1>Modifiez votre Pokémon<span>.</span></h1>
        <p>Retrouvez ses informations et enregistrez vos changements.</p>
    </div>

    <?php require __DIR__ . '/form.php'; ?>

    <footer class="create-footer">PokeClub <span aria-hidden="true">·</span> À chaque dresseur,
    son équipe.</footer>
</main>

<?php require __DIR__ . '/../footer.php'; ?>
```

Le bloc d'introduction affiche le titre et le lien de retour. L'inclusion de form.php place les champs dans la page. L'inclusion finale termine le document.

create.php suit ce principe avec une mise en page plus développée : un panneau de conseils, une Poké Ball dessinée en SVG, puis le formulaire. Les balises svg, circle et path décrivent le dessin ; leurs coordonnées et leurs couleurs règlent son apparence.

Le contrôleur gère le traitement des données. Ces vues organisent les éléments affichés autour du formulaire partagé.

form.php : l’adresse d’envoi et les messages

src/View/pokemon/form.php

Le même formulaire sert à créer ou modifier un Pokémon, grâce aux variables fournies par le contrôleur.

Bloc du projet : src/View/pokemon/form.php | lignes 8 à 25

```
<form action="<?= e($formAction) ?>" method="post" class="create-form">
    <input type="hidden" name="csrf_token" value="<?= e(csrfToken()) ?>" />
    <?php if ($success !== null) { ?>
        <div class="create-section" role="status">
            <h2 class="font-bold">Pokémon enregistré !</h2>
            <p><?= e($success) ?> : <?= e($successMessage) ?></p>
        </div>
    <?php } ?>
    <?php if ($errors !== []) { ?>
        <div class="create-section" role="alert">
            <h2 class="font-bold">Le Pokémon n’a pas été enregistré.</h2>
            <ul class="list-disc pl-5">
                <?php foreach (array_unique($errors) as $error) { ?>
                    <li><?= e($error) ?></li>
                <?php } ?>
            </ul>
        </div>
    <?php } ?>
```

`action` utilise $formAction : la route de création ou la route d’édition avec son identifiant. `method="post"` fait arriver les champs dans $_POST. Le champ caché transmet le jeton CSRF.

Le premier if affiche la confirmation lorsque $success n’est pas null. Le second affiche les erreurs lorsqu’il y en a. array_unique() retire les doublons avant l’affichage ; le contrôleur actuel interrompt la validation à la première erreur. e() prépare chaque texte pour le HTML.

Un champ HTML : comprendre name, id et value

src/View/pokemon/form.php

Bloc du projet : src/View/pokemon/form.php | lignes 36 à 38

```
<label for="name">Nom du Pokémon</label>
<input type="text" id="name" name="name" placeholder="Ex. : Gardevoir"
    value="<?= e($values['name']) ?>" maxlength="255" required />
```

name="name"	C’est la clé envoyée au serveur. Le contrôleur retrouvera sa valeur dans $_POST['name'].
id="name" et for="name"	Ils relient le champ et son libellé. Cliquer le libellé peut ainsi donner le focus au champ.
value="..."	Affiche le contenu courant de $values['name'] : valeur initiale, donnée de la base ou saisie conservée.
placeholder	Montre un exemple lorsque le champ est vide. Ce texte n’est pas la valeur envoyée.
maxlength et required	Limitent et exigent la saisie dans le navigateur. Les validations PHP vérifient aussi les données reçues côté serveur.

Le champ d'image fonctionne sur le même principe avec type="url" et la clé picture. item reçoit l'objet tenu. Les champs move-1 à move-4 reçoivent les attaques ; seul move-1 est obligatoire dans le parcours du formulaire.

Les listes déroulantes : cases(), value et label()

src/View/pokemon/form.php

Bloc du projet : src/View/pokemon/form.php | lignes 41 à 46

```
<label for="nature">Nature</label>
<select id="nature" name="nature">
    <?php foreach (PokemonNature::cases() as $nature) { ?>
        <option value="<?= e($nature->value) ?>" <?= $values['nature'] === $nature->value ? '
    selected' : '' ?>><?= e($nature->label()) ?></option>
    <?php } ?>
</select>
```

PokemonNature::cases() retourne tous les cas possibles. La boucle produit une balise option par cas. ->value fournit la chaîne envoyée au serveur et label() fournit le libellé affiché.

Le test sur $values['nature'] ajoute selected à l'option correspondant à la valeur actuelle. Le même fonctionnement est utilisé pour type-1 et type-2 avec PokemonType::cases(). La deuxième liste possède aussi une option vide « Aucun ».

Exemple pour comprendre

Pour une nature modeste, l'option envoie le texte modeste, affiche Modeste et devient sélectionnée si $values['nature'] vaut modeste.

Les statistiques et le bouton d'envoi

src/View/pokemon/form.php

Bloc du projet : src/View/pokemon/form.php | lignes 121 à 123

```
<div class="create-field"><label for="hp"><span class="create-stat-dot stat-hp"
            aria-hidden="true"></span>PV</label><input type="number" id="hp" name="hp"
        placeholder="0" value="<?= e($values['hp']) ?>" min="0" max="32" step="1" /></div>
```

Les six groupes de champs suivent ce modèle : hp, atk, def, spa, spd et spe. Le navigateur propose des entiers de 0 à 32 avec step="1". La valeur affichée vient de $values et le modèle vérifie aussi les limites et le total.

Le bouton type="submit" envoie le formulaire. Son libellé vient de $submitLabel, ce qui permet d'afficher « Créer mon Pokémon » ou « Enregistrer les modifications ». Le lien d'annulation ramène simplement à l'accueil.

home.php : le résumé et les places de l’équipe

src/View/home.php

Cette vue reçoit $team, ses membres dans $pokemons et les types distincts dans $teamTypes.

Bloc du projet : src/View/home.php | lignes 22 à 24

```
<?php for ($slot = 0; $slot < 6; $slot++) { ?>
    <span class="home-slot<?= $slot < count($pokemons) ? ' is-filled' : '' ?>"></span>
<?php } ?>
```

La boucle for dessine six places. Elle ajoute is-filled aux places déjà occupées. Les compteurs affichent ensuite le nombre de membres, le nombre de types distincts et les places restantes avec max(0, 6 - count($pokemons)).

Si $pokemons est vide, un bloc invite à créer un Pokémon. Sinon, le bloc de cartes parcourt les associations de l’équipe. Le nom de l’équipe vient de getName(), avec un texte d’attente lorsqu’aucune équipe n’a été chargée.

home.php : une carte par association

src/View/home.php

Bloc du projet : src/View/home.php | lignes 67 à 68

```
$pokemon = $pokemonTeam->getPokemon();
$points = $pokemon->getEffortPoints();
```

Chaque $pokemonTeam est une association, pas directement un Pokemon. getPokemon() retrouve le modèle Pokémon ; getRole() fournit son rôle dans cette équipe. L’indice sert à créer des identifiants HTML différents pour les cartes.

Les autres blocs de la carte affichent l’image si elle existe, le nom, les types, l’objet tenu, la nature, les attaques et les statistiques. Les getters lisent l’objet et e() prépare les textes pour le HTML. Les listes de types et d’attaques utilisent chacune leur foreach.

Le lien « Modifier ce Pokémon » contient son identifiant et conduit à l’action edit(). Le lien « Voir tous les Pokémon » conduit actuellement au fichier list.php, encore vide.

Les pages publiques anciennes ou incomplètes

public/pokepage/edit.php | public/pokepage/list.php | public/pokepage/show.php

Bloc du projet : public/pokepage/edit.php | lignes 1 à 9

```
<?php

$id = $_GET['id'] ?? '';
$query = ['page' => 'pokemon-edit'];
if (is_string($id)) {
    $query['id'] = $id;
}
header('Location: /?' . http_build_query($query), true, 303);
exit;
```

Ce fichier edit.php conserve une ancienne adresse. Il récupère id lorsqu’il est une chaîne, prépare la route pokemon-edit et utilise http_build_query() pour construire les paramètres de l’URL. header() redirige vers

index.php ; exit termine cette ancienne entrée.

public/pokepage/list.php est vide : la liste de collection reste à créer. public/pokepage/show.php inclut seulement header.php et footer.php : la lecture et l'affichage d'une fiche de détail ne sont pas encore réalisés.

10. Les autres modèles du projet

Ces classes utilisent les mêmes mécanismes d'objets et d'accès aux propriétés. Elles constituent des données déjà définies, avec des parcours de pages encore à compléter.

Le modèle commun : constructeur, getters et setters

src/Model/Game.php | src/Model/OppoTeam.php | src/Model/ProPlayer.php

Ces classes conservent un identifiant et des caractéristiques. Le constructeur initialise les informations ; les getters les lisent ; les setters les remplacent.

Bloc du projet : src/Model/OppoTeam.php | lignes 10 à 31

```
public function __construct(string $name)
{
    $this->name = $name;
}

public function getId(): ?int
{
    return $this->id;
}
public function setId(int $id): void
{
    $this->id = $id;
}

public function getName(): string
{
    return $this->name;
}
public function setName(string $name): void
{
    $this->name = $name;
}
```

Dans cet exemple, le constructeur conserve le nom de l'équipe adverse. getName() le retourne. setName() reçoit un nouveau texte et l'affecte à la propriété. getId()/setId() font la même chose pour l'identifiant, initialement null.

Ces méthodes simples ne contiennent pas de validation métier supplémentaire dans la version actuelle. Elles changent ou lisent l'objet en mémoire.

Game : les informations d’un match

src/Model/Game.php

Le constructeur	Reçoit un objet DateTime pour la date, une chaîne pour la ville et une chaîne pour le tournoi.
getId() / setId()	Lisent ou attribuent l’identifiant, initialement null.
getDate() / setDate()	Lisent ou remplacent l’objet de date.
getCity() / setCity()	Lisent ou remplacent le texte de la ville.
getTournament() / setTournament()	Lisent ou remplacent le texte du tournoi.

Cette classe ne calcule pas encore de score et ne contient pas de composition de manches. La notation datetime utilisée dans le fichier désigne la classe DateTime importée au début.

ProPlayer : une fiche de joueur professionnel

src/Model/ProPlayer.php

Le constructeur	Reçoit firstname, lastname, nationality et picture, puis conserve ces quatre textes.
getId() / setId()	Accèdent à l’identifiant.
getFirstname() / setFirstname()	Accèdent au prénom.
getLastname() / setLastname()	Accèdent au nom de famille.
getNationality() / setNationality()	Accèdent à la nationalité.
getPicture() / setPicture()	Accèdent à l’adresse de l’image.

La structure des méthodes est celle de l’exemple OppoTeam. Cette classe reste indépendante des équipes dans le code étudié : aucun lien supplémentaire n’est créé dans ces méthodes.

GameRound.php : un fichier prévu

src/Model/GameRound.php

Le fichier est actuellement vide. Il ne contient donc ni classe, ni constructeur, ni méthode de gestion des manches à expliquer.

Ce statut est distinct de Game.php, où des propriétés et des méthodes existent déjà. La gestion complète des matchs n’est pas encore reliée aux routes du site.

11. Composer, CSS et jeu d’essai

Ces fichiers préparent les bibliothèques, la présentation et les exemples de fonctionnement. Ils accompagnent le code PHP des pages.

composer.json : déclarer les besoins PHP

composer.json

Composer lit ce fichier pour savoir quelles bibliothèques installer et comment charger les classes du projet.

Bloc du projet : composer.json | lignes 5 à 15

```
"require": {
    "php": "^8.3",
    "ext-pdo": "*",
    "ext-pdo_mysql": "*",
    "ext-mbstring": "*",
    "ext-session": "*",
    "ext-filter": "*",
    "ext-json": "*",
    "monolog/monolog": "^3",
    "symfony/validator": "^7.4"
},
```

La version de PHP et les extensions sont des exigences pour l’environnement. Composer vérifie leur présence ; les bibliothèques déclarées ici sont Monolog et Symfony Validator.

Monolog sert au journal et Symfony Validator aux contraintes de nom et d’URL. Le symbole ^ demande une plage compatible : ^3 pour Monolog permet les versions 3.x, et ^7.4 de Symfony demande au moins 7.4 avant la version majeure 8. Les champs name, description, type, license et authors décrivent le projet.

Le bloc autoload

Bloc du projet : composer.json | lignes 23 à 27

```
"autoload": {
    "psr-4": {
        "PokeClub\\": "src/"
    }
}
```

La correspondance PSR-4 dit à Composer que le préfixe PokeClub\ commence dans src/. Ainsi, PokeClub\Controller\PokemonController correspond au fichier src/Controller/PokemonController.php.

composer.lock fixe les versions exactes retenues. composer install les réinstalle et génère le chargeur dans vendor/. bootstrap.php inclut vendor/autoload.php avant l’utilisation des classes.

Comprendre les fichiers ouverts dans vendor/

vendor/autoload.php	Entrée du chargement automatique de Composer.
autoload_real.php	Initialise le chargeur, ses correspondances et les fichiers à inclure.
autoload_psr4.php	Associe des préfixes de namespace à des dossiers.
autoload_namespaces.php	Contient les éventuelles correspondances de l'ancien chargement PSR-0.
autoload_files.php	Liste les fichiers de fonctions à inclure automatiquement.
Symfony Constraints/Count.php	Définit une contrainte sur un nombre d'éléments. Elle appartient à la bibliothèque installée et n'est pas appelée par le code de l'application étudié.

Les noms de bibliothèques écrits dans les use sont des raccourcis de noms. C'est l'autoloader qui retrouve leur code au moment où PHP en a besoin. Les fichiers de vendor/ sont produits ou installés par Composer.

package.json : préparer les commandes Tailwind

package.json

Bloc du projet : package.json | lignes 4 à 11

```
"scripts": {
  "dev": "tailwindcss -i ./src/assets/css/tailwind.css -o ./public/assets/css/tailwind.css
    --watch",
  "build": "tailwindcss -i ./src/assets/css/tailwind.css -o ./public/assets/css/tailwind.css
    --minify"
},
"devDependencies": {
  "@tailwindcss/cli": "^4.3.3",
  "tailwindcss": "^4.3.3"
}
```

Le bloc scripts définit les commandes npm run dev et npm run build. Elles lisent src/assets/css/tailwind.css et écrivent le résultat dans public/assets/css/tailwind.css.

--watch surveille les modifications pendant le développement. --minify réduit la taille du CSS produit. devDependencies déclare Tailwind et son outil de ligne de commande. Le champ private: true réserve le paquet au projet et empêche sa publication npm.

Le CSS par familles de blocs

src/assets/css/tailwind.css

La feuille de style complète Tailwind avec les classes de présentation utilisées dans les vues.

Import et thème	@import charge Tailwind. Les variables --create-ink, --create-muted et --create-accent fournissent les couleurs réutilisées avec var().
Cadre de la page	Les classes create-container, create-header, create-brand et create-nav règlent la largeur, le logo et la navigation.

Page et formulaire	create-layout organise les colonnes. create-section et create-fields organisent les groupes de champs ; create-stats dispose les statistiques.
États des champs et boutons	:hover réagit au survol, :focus au focus et :focus-visible au focus à rendre visible. Ces blocs modifient bordures, couleurs ou contours.
Accueil et cartes	Les classes home-* règlent les compteurs, les places, la grille des membres et les cartes Pokémon.
Adaptation aux écrans	Les blocs @media modifient colonnes, espacements et éléments visibles selon la largeur. Le dernier retire les transitions si les animations réduites sont demandées.

Bloc du projet : src/assets/css/tailwind.css | lignes 31 à 31

```
.create-layout { display: grid; grid-template-columns: 290px minmax(0, 1fr); align-items: start;
    gap: 32px; }
```

Ce bloc cible les éléments portant la classe create-layout. display: grid choisit une grille. grid-template-columns réserve 290 pixels au panneau et donne le reste au formulaire avec minmax(0, 1fr). gap fixe l’espace entre les deux.

Une classe HTML se retrouve grâce au point dans le sélecteur CSS : class="create-layout" correspond à .create-layout. Les règles @media plus bas adaptent cette mise en page aux petits écrans. Le fichier chargé par header.php est le résultat généré dans public/.

fixtures.php : essayer les objets dans le terminal

src/scripts/fixtures.php

Ce script construit un exemple de Pokémon et affiche ses caractéristiques. Il permet d’observer les objets sans passer par les pages.

Bloc du projet : src/scripts/fixtures.php | lignes 8 à 18

```
$pokemon = new Pokemon('Rampe-Aile', ['Insecte', 'Combat'], ['Close Combat', 'Seisme',
    'Demi-tour', 'Abri'], 'Bandeau Choix', [2, 32, 0, 0, 0, 32], 'adamant', '');
echo ('Le pokémon du jour est : ') . $pokemon->getName() . (', il est de type : ') .
    implode('/', array_map(fn(PokemonType $type) => $type->label(), $pokemon->getTypes())) . ('
    et sa nature est la suivante : ') . $pokemon->getNature()->label() . PHP_EOL;
echo ('Ses attaques sont les suivantes : ') . PHP_EOL;
foreach ($pokemon->getMoves() as $move) {
    echo $move . PHP_EOL;
}
echo ('Il tient un objet qui est : ') . $pokemon->getItem() . PHP_EOL;
echo ('Ses statistiques sont réparties de la manière suivante : ') . PHP_EOL;
foreach ($pokemon->getEffortPoints() as $key => $value) {
    echo ('Pour la statistique de ' . $key . ', on distribue le nombre de points suivant :' .
    $value) . PHP_EOL;
}
```

Le premier bloc new Pokemon(...) passe par les mêmes setters que le formulaire. Les types sont reconnus malgré les majuscules et la nature anglaise adamant est convertie en rigide.

array_map() applique la petite fonction fn à chaque type pour obtenir son libellé. implode('/', ...) assemble ces textes. Les foreach affichent ensuite les attaques et les statistiques. PHP_EOL ajoute les retours à la ligne du terminal.

Le lancement se fait avec php src/scripts/fixtures.php. Ce script charge d'abord l'autoloader ; il ne fait aucun appel à un repository de sauvegarde.

pokeclub.sql : le schéma à compléter

src/sql/pokeclub.sql

Ce fichier est actuellement vide. Il ne contient pas encore les instructions CREATE TABLE pour recréer la base.

Les repositories utilisent déjà les tables pokemon, team et pokemon_team présentes dans la base configurée. Le fichier SQL du projet reste à remplir pour documenter et reproduire ce schéma.

12. Suivre un parcours de bout en bout

Ces trois exemples permettent de relier les blocs au lieu de les apprendre séparément.

Créer un Pokémon valide

1. Le navigateur demande /?page=pokemon-create. index.php prépare les objets puis appelle create().

2. GET saute le traitement POST. defaultValues() fournit les champs et la vue affiche le formulaire.

3. À l'envoi POST, verifyCsrf() vérifie le jeton. readForm() retire les espaces des textes.

4. createPokemon() vérifie le nom et l'image, convertit les statistiques et prépare les tableaux de types et d'attaques.

5. new Pokemon(...) appelle les setters. Si les règles métier sont respectées, l'objet revient au contrôleur.

6. save() demande au mapper le tableau SQL. Comme l'id est null, le repository insère une ligne et attribue l'identifiant créé.

7. Le contrôleur conserve le nom en session, redirige en 303 puis termine le POST. Le nouveau GET affiche la confirmation.

Refuser une statistique à 33

1. readForm() conserve d'abord le texte '33'. filter_var() le convertit en entier : la conversion elle-même est valide.

2. Le constructeur de Pokemon arrive à setEffortPoints(). La borne de 32 est dépassée : le modèle lance une InvalidArgumentException.

3. L'exception remonte jusqu'au catch de create() ou edit(). Son message est placé dans $errors. save() n'est pas exécutée dans ce parcours.

4. Le formulaire est réaffiché avec le statut 422 et les valeurs préparées de cet envoi ordinaire, pour corriger la statistique.

Modifier la fiche 42

1. edit() vérifie id=42 et appelle findById(42). Le mapper reconstruit le Pokemon existant.

2. Les getters servent à préremplir le formulaire. Le navigateur envoie les changements vers la même route avec id=42.

3. La nouvelle saisie produit un nouvel objet en mémoire. setId(42) lui redonne l'identité de la fiche chargée.

4. save() voit cet identifiant et choisit UPDATE avec WHERE id = :id. La fiche 42 est modifiée.

5. La confirmation et la redirection reprennent le même principe qu'en création.

Les rôles à distinguer pendant ta lecture

Lire les champs	readForm() récupère du texte depuis $_POST.
Créer l'objet	createPokemon() et le constructeur de Pokemon produisent et vérifient un objet en mémoire.
Enregistrer	PokemonRepository::save() exécute l'écriture SQL.
Afficher	Les vues insèrent les valeurs et les messages dans du HTML.

Pour comprendre un appel, pose toujours ces trois questions : à quel objet appartient la méthode, quelles données reçoit-elle et quel résultat ou effet produit-elle ? Tu peux ensuite suivre le fichier correspondant dans ton éditeur.